

Sistema de Gestão de Estoque de Reagentes de Laboratório

Grupo 10:

Gustavo Santos Pinto

Marcus Vinicius de Souza Santos Prado

Nathalia de Assis Almeida

Rafael Rangel Fietto

Tais dos Santos Ferreira

Índice

- **Motivação**
- **Tipos de usuário**
- **ODS**
- **Herança**
- **Polimorfismo**

Motivação

O laboratório de Bioquímica enfrentava dificuldades para organizar e controlar a retirada de reagentes, o que resultava em registros incompletos, risco de falta de materiais e pouca visibilidade sobre o uso diário. Para solucionar esse problema, desenvolvemos um sistema integrado capaz de gerenciar reagentes, acompanhar retiradas e centralizar todas as informações em um único ambiente, garantindo mais controle, segurança e eficiência no funcionamento do laboratório.

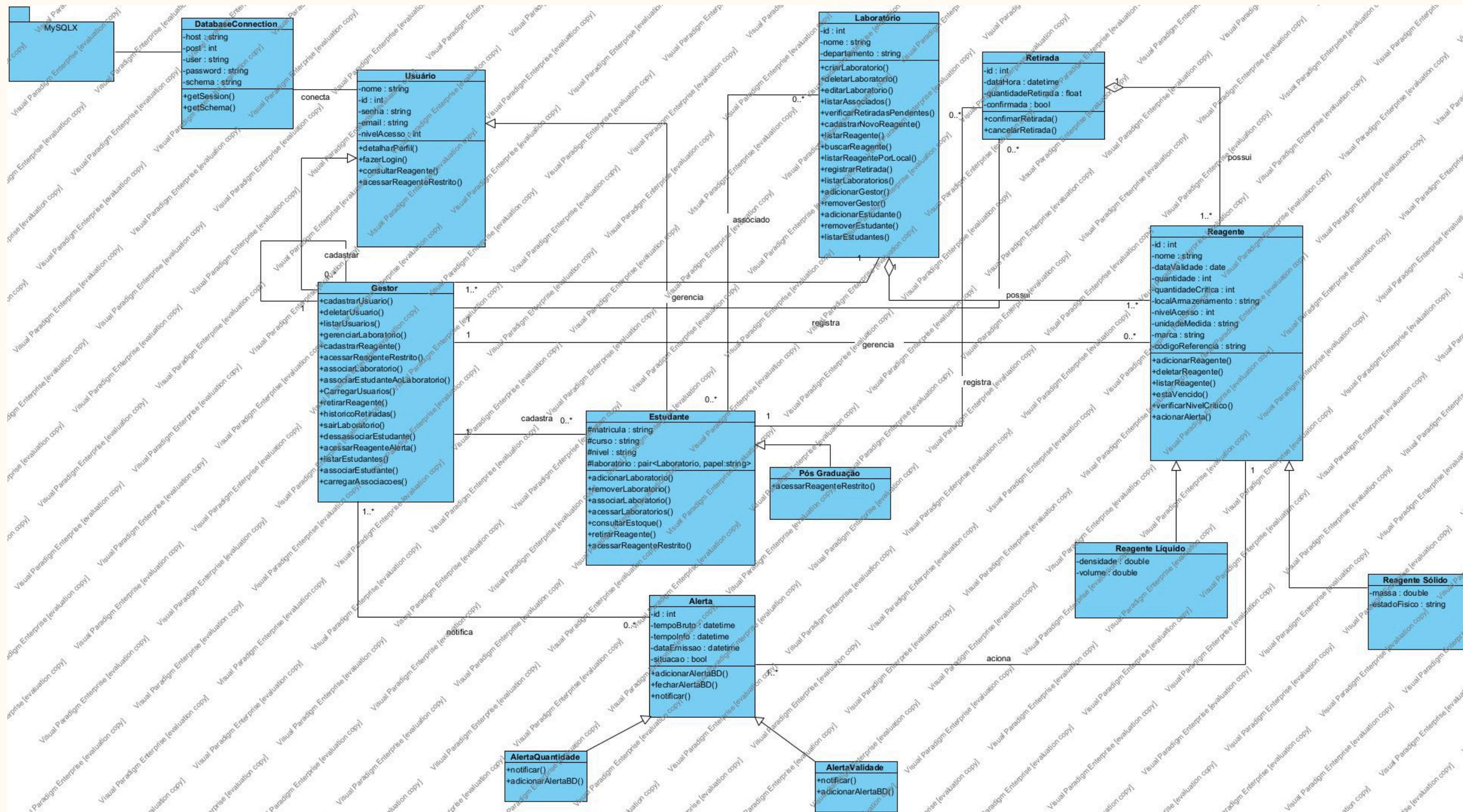
Tipos de Usuários

- Gestores
- Estudantes (Graduação)
- Pós-Graduação

Contribuição para os ODS

- **ODS 3 – Saúde e Bem-Estar: Previne o uso de reagentes vencidos, aumentando a segurança laboratorial.**
- **ODS 9 – Indústria, Inovação e Infraestrutura: Soluções tecnológicas para otimização de processos.**
- **ODS 12 – Consumo e Produção Responsáveis: Uso racional de reagentes e descarte adequado.**

Diagrama de Classes



Herança

Usuário → Classe Mãe

```
class Usuario{
protected:
    Schema *db; // Ponteiro para o esquema do banco de dados
private:
    std::string senha; // Senha do Usuario
    std::string nome; // Nome Completo do Usuario
    std::string email; // E-mail do Usuario
    int id; // ID do Usuario, por normalização, o ID inicial será -1, significa que não
    // cadastrado ou encontrado, assim que algumas dessas sentenças for verdadeiras, o BD retorna s
    int nivelAcesso; // Corresponde ao Tipo de Usuário, devido que a depender possui influência no
    Gestor * cadastradoPor;
```

Gestor → Classe filha

```
class Gestor : public Usuario
{
private:
    // O laboratorio que este Gestor gerencia
    Laboratorio *laboratorio;

public:
    // Construtor
    Gestor(std::string nome, std::string email, std::string senha, int nivelAcesso, Schema *db);
```

Estudante → Classe filha

```
class Gestor : public Usuario
{
private:
    // O laboratorio que este Gestor gerencia
    Laboratorio *laboratorio;

public:
    // Construtor
    Gestor(std::string nome, std::string email, std::string senha, int nivelAcesso, Schema *db);
```

Herança

Reagente → Classe Mãe

```
class Reagente
{
private:
    std::string nome;
    std::string dataValidade;
    int quantidade;
    int quantidadeCritica;
    std::string localArmazenamento;
    int nivelAcesso;
    std::string unidadeMedida;
    std::string marca;
    std::string codigoReferencia;
    int id;
```

Reagente liquido → Classe filha

```
class ReagenteLiquido : public Reagente {
private:
    double densidade;
    double volume;

public:
    //Construtor
    ReagenteLiquido(int id, std::string nome, std::string dataValidade, int quantidade,
                    int quantidadeCritica, std::string localArmazenamento, int nivelAcesso,
                    std::string unidadeMedida, std::string marca, std::string codigoReferencia,
                    double densidade, double volume);

    //Destrutor
    ~ReagenteLiquido() override; // subrescrevendo o metodo virtual
```

Reagente sólido → Classe filha

```
class ReagenteSolido : public Reagente {
private:
    double massa;
    std::string estadoFisico;

public:
    //Construtor
    ReagenteSolido(int id, std::string nome, std::string dataValidade, int quantidade,
                    int quantidadeCritica, std::string localArmazenamento, int nivelAcesso,
                    std::string unidadeMedida, std::string marca, std::string codigoReferencia,
                    double massa, std::string estadoFisico);

    //Destrutor
    ~ReagenteSolido() override; //sobrescrevendo o metodo virtual
```

Herança

Alerta → Classe Mãe

```
class Alerta
{
protected:
    int _id;
    time_t _tempoBruto;      // tempo em unix time stamp
    struct tm *_tempoInfo;   // struct que guarda informacoes do tempo atual
    std::string _dataEmissao; // tempo formatado para leitura
    bool _situacao;         // aberto ou fechado
    Reagente *_reagenteEmAlerta;
```

Alerta quantidade → Classe filha

```
class AlertaQuantidade : public Alerta
{
public:
    AlertaQuantidade(Reagente *reagenteEmAlerta, bool situacao);
    AlertaQuantidade(int id, Reagente *reagenteEmAlerta, std::string dataEmissao, bool situacao);

    void notificar() override;
    void adicionarAlertaBD(Schema *db) override;
};
```

Alerta Validade → Classe filha

```
class AlertaValidade : public Alerta
{
public:
    AlertaValidade(Reagente *reagenteEmAlerta, bool situacao);
    AlertaValidade(int id, Reagente *reagenteEmAlerta, std::string dataEmissao, bool situacao);

    void notificar() override;
    void adicionarAlertaBD(Schema *db) override;
};
```


Polimorfismo

Classe Alerta

```
virtual void adicionarAlertaBD(Schema *db) = 0;  
void fecharAlertaBD();  
virtual void notificar() = 0;
```

Alerta quantidade

```
class AlertaQuantidade : public Alerta  
{  
  
public:  
    AlertaQuantidade(Reagente *reagenteEmAlerta, bool situacao);  
    AlertaQuantidade(int id, Reagente *reagenteEmAlerta, std::string dataEmissao, bool situacao);  
  
    void notificar() override;  
    void adicionarAlertaBD(Schema *db) override; ←  
};
```

Alerta validade

```
class AlertaValidade : public Alerta  
{  
  
public:  
    AlertaValidade(Reagente *reagenteEmAlerta, bool situacao);  
    AlertaValidade(int id, Reagente *reagenteEmAlerta, std::string dataEmissao, bool situacao);  
  
    void notificar() override;  
    void adicionarAlertaBD(Schema *db) override; ←  
};
```